

Code Migration: Streamlining with Generative AI

Code Migration is the process of moving an application from one language, framework, platform, or version to another while preserving functionality. Traditionally, migration is time-consuming and error-prone. **Generative AI (GenAI)** significantly accelerates and simplifies this process.

1. Code Migration

Code migration involves:

- Converting source code between programming languages
- Moving applications to new frameworks
- Upgrading runtime versions
- Refactoring legacy code

Why Migration is Needed

- Technology becomes outdated
 - Security vulnerabilities
 - Performance improvements
 - Cloud adoption
-

2. Types of Code Migration

Type	Description
Language Migration	Java → Python
Framework Migration	AngularJS → React
Platform Migration	On-prem → Cloud
Version Upgrade	Python 2 → Python 3
Architecture Migration	Monolith → Microservices

3. Generative AI in Code Migration: Advantages

- Faster conversion
 - Automated refactoring
 - Consistent coding style
 - Reduced manual effort
 - Lower cost
-

4. Generative AI Tools for Code Migration

- ChatGPT
 - GitHub Copilot
 - Amazon CodeWhisperer
 - Google Gemini Code Assist
 - Tabnine
-

5. Language Conversion with Generative AI

Example: Java → Python

Java Code

```
int sum = a + b;
```

GenAI Output

```
sum = a + b
```

6. Steps in Language Conversion Without Generative AI

1. Analyze source code
2. Learn target language
3. Manually rewrite code
4. Fix syntax errors
5. Test

Time-consuming

7. Steps in Language Conversion With Generative AI

1. Paste code into GenAI
2. Ask for conversion
3. Review output
4. Test

Fast

8. Demo: Facilitating Language Conversion Using Generative AI

Prompt

Convert this Java login program to Python

GenAI Output

```
def login(username, password):  
    if username == "admin" and password == "123":  
        return "Success"  
    return "Failed"
```

9. Framework Migration

Moving from one framework to another.

Example:

- Flask → FastAPI
 - AngularJS → React
-

Steps for Framework Migration Without Generative AI

1. Learn new framework
2. Rewrite controllers
3. Rewrite views
4. Rewrite routing
5. Test

Steps for Framework Migration With Generative AI

1. Provide old code
 2. Ask GenAI to convert
 3. Review & adjust
 4. Test
-

Demo:

Implementing Framework Migration Using Generative AI

Prompt

Convert Flask REST API to FastAPI

FastAPI Output

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello"}
```

Version Upgrades

Updating language/framework version.

Example:

- Python 2 → Python 3
 - Node 10 → Node 18
-

Steps in Version Upgrades Without Generative AI

1. Read migration docs
 2. Identify breaking changes
 3. Update code manually
 4. Test
-

Steps in Version Upgrades With Generative AI

1. Paste code
 2. Ask GenAI to modernize
 3. Review
 4. Test
-

Version Upgrades: Example

Old Python 2

```
print "Hello"
```

AI Output

```
print("Hello")
```

Limitation of GenAI

- May generate incorrect code
 - Needs human review
 - Limited business context
 - Security concerns
-

Use Case: Uber's Code Migration and Refactoring

Uber migrated large monolithic services to microservices and modern frameworks.

GenAI-like tools help by:

- Refactoring legacy code
- Converting services
- Generating tests

Result

- Faster migration
 - Improved scalability
 - Better maintainability
-

Quick Check

Q1. What is code migration?

- ✓ Moving code to new language/framework/version

Q2. True/False

GenAI eliminates testing.

False

Q3. Which is faster?

- A) Manual conversion
- B) GenAI-assisted conversion

- ✓ B

Q4. Give one limitation

- ✓ Needs human review
-

Summary

Generative AI simplifies and accelerates:

- Language conversion
- Framework migration
- Version upgrades

When combined with testing and human oversight, GenAI becomes a **powerful migration accelerator**.
